

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт

з лабораторної роботи №1

з дисципліни: «Поглиблене вивчення JavaScript»

з теми: «Розробка базового застосунку керування задачами (To-Do List)»

Виконав:

ст. гр. ПЗПІ-23-2

Малишкін Андрій

Перевірів:

посада викладача (уточнити)

ПІБ викладача (уточнити)

1 РОЗРОБКА БАЗОВОГО ЗАСТОСУНКУ КЕРУВАННЯ ЗАДАЧАМИ (TO-DO LIST)

1.1 Мета роботи

Метою роботи є ознайомлення з основними принципами роботи фреймворку Vue 3, формування практичних навичок створення односторінкових застосунків та засвоєння механізмів реактивності, директив, обробки подій і компонентного підходу.

1.2 Завдання

У межах лабораторної роботи необхідно:

- створити проєкт із використанням Vue 3;
- реалізувати введення нових задач;
- забезпечити додавання задач до списку;
- організувати відображення списку задач;
- реалізувати зміну стану задач;
- реалізувати видалення задач;
- додати фільтрацію задач;
- за можливості реалізувати компонентну структуру застосунку.

1.3 Хід роботи

1.3.1 Теоретичні відомості

Було розглянуто, що Vue.js використовує компонентний підхід до побудови інтерфейсів, за якого логіка застосунку поділяється на автономні частини. Також було опрацьовано механізм реактивності: під час зміни значень стану інтерфейс автоматично синхронізується без ручної роботи з DOM.

Окрему увагу було приділено директивам шаблонів («v-model», «v-for», умовним виразам) та подієвій моделі («@click», «@submit», «@change»), які забезпечують зв'язок між діями користувача і логікою компонента.

1.3.2 Етапи реалізації застосунку

Під час практичної реалізації було послідовно виконано такі етапи:

- підготовлено проектну структуру Vue 3 + TypeScript у директорії «src»;
- видалено типовий стартовий шаблон та сформовано базову структуру інтерфейсу;
- реалізовано форму введення задач із перевіркою порожнього вводу;
- реалізовано додавання нових задач у реактивний список;
- реалізовано зміну стану задач (виконано/активно);
- реалізовано видалення задач зі списку;
- реалізовано фільтрацію задач за трьома режимами відображення;
- оформлено інтерфейс у темній темі з використанням Tailwind CSS та shadcn-подібних UI-компонентів.

1.3.3 Опис архітектури та компонентів

Застосовано компонентну декомпозицію:

- «App.vue»: керування станом, фільтрами та статистикою;
- «TodoList.vue»: відображення відфільтрованої колекції задач;
- «TodoItem.vue»: керування окремою задачею (toggle, remove);
- «UiButton.vue», «UiInput.vue», «UiCard.vue»: базові UI-примітиви для уніфікації вигляду елементів.

Такий поділ дозволив ізолювати відповідальність компонентів і спростити подальшу підтримку коду.

1.3.4 Короткі фрагменти програмного коду

Нижче наведено ключові фрагменти реалізації, що демонструють основні принципи роботи застосунку.

1.3.4.1 Додавання задачі з валідацією

```
function addTodo(): void {
  const normalizedTitle = inputValue.value.trim()
  if (!normalizedTitle) return

  todos.value.unshift({
    id: Date.now(),
    title: normalizedTitle,
    completed: false,
  })
}
```

```
    inputValue.value = ''
  }
```

1.3.4.2 Фільтрація задач через обчислюване значення

```
const visibleTodos = computed(() => {
  if (currentFilter.value === 'active') {
    return todos.value.filter((todo) => !todo.completed)
  }
  if (currentFilter.value === 'completed') {
    return todos.value.filter((todo) => todo.completed)
  }
  return todos.value
})
```

1.4 Результати роботи

У результаті виконання лабораторної роботи отримано працездатний застосунок керування задачами. Було забезпечено повний цикл роботи зі списком задач: додавання, відображення, зміну стану, видалення та фільтрацію. Також реалізовано адаптивний інтерфейс із темною темою та уніфікованими елементами керування.

Під час перевірки працездатності було підтверджено:

- коректність додавання задач після валідації вводу;
- коректність роботи перемикачів статусу задач;
- коректність видалення задач і перерахунку статистики;
- коректність фільтрації за всіма режимами відображення;
- відсутність помилок lint/typescheck у проєкті.

1.5 Висновки

Під час виконання лабораторної роботи було закріплено практичні підходи до розробки SPA на Vue 3. Було відпрацьовано застосування реактивності, директив та компонентного підходу, а також отримано навички структурованої організації фронтенд-коду з використанням TypeScript.

1.6 Контрольні запитання

а) Що таке SPA?

SPA — це односторінковий застосунок, у якому оновлення інтерфейсу виконується без повного перезавантаження сторінки.

б) Основні принципи роботи Vue.js.

Основними принципами Vue.js є реактивність даних, декларативний опис інтерфейсу, компонентний підхід та подієва взаємодія елементів інтерфейсу.

в) Що таке реактивність?

Реактивність — це механізм автоматичного відстеження змін стану з подальшим оновленням пов'язаних елементів інтерфейсу.

г) Яке призначення директив?

Директиви використовуються для зв'язку шаблону з даними, умовного відображення елементів, ітерації по колекціях та обробки подій.

д) Як реалізується обробка подій?

Обробка подій реалізується прив'язкою подій користувача до відповідних методів або функцій компонента.

е) Що таке компонент?

Компонент — це самостійна частина застосунку, що містить шаблон, логіку та стилі та може використовуватися повторно.